Name : Vihan D. I.

Reg No : EG/2020/4323

EC8207 Applied Big Data Engineering
Mini Project Assessment - Project Report

FinTech Fraud Detection Pipeline

1. Introduction

This report describes the design and implementation of a real-time fraud detection pipeline built for the FinTech domain. The system ingests financial transaction data from a Kafka message broker, applies stateful fraud detection rules using Apache Spark Structured Streaming, stores results to a PostgreSQL database and runs a scheduled ETL process pipeline via Apache Airflow every six hours. The pipeline follows a Lambda architecture, combining a speed layer for low-latency fraud flagging with a batch layer for storing the validated data in a data warehouse and generating reconciliation and analysis reports. Grafana is used to implement a live monitoring dashboard which uses PostgreSQL as its data source to provide an overview of analytical results of the pipeline.

2. Architecture

Lambda architecture was selected for developing the pipeline. The pipeline consists of five core components. Apache Kafka acts as the ingestion layer where it receives transaction events from the producer. Apache Spark Structured Streaming forms the speed layer where it consumes from Kafka in real time and applies stateful fraud detection rules before writing results to PostgreSQL database. Apache Airflow ETL pipeline runs every six hours as the batch layer where it extracts validated transactions into a data warehouse and then generates a reconciliation report. Finally, Grafana provides a live monitoring dashboard for observing the real time analytical results of the pipeline.

The reasoning for choosing the Lambda architecture is that the reconciliation reporting requirement introduces a feature that is best implemented as a batch workload rather than using a streaming implementation where recomputing aggregate statistics over a six-hour window is more efficient in the manner of a bounded batch job than as an unbounded stream. Additionally, the validated Parquet file data warehouse acts as a historical record of cleaned data that is in the form of periodic, consistent snapshots rather than continuous incremental updates. Kappa architecture eliminates the batch layer entirely and replaces it with stream replay which would add complexity to the reconciliation process without providing any additional benefit for this scenario.

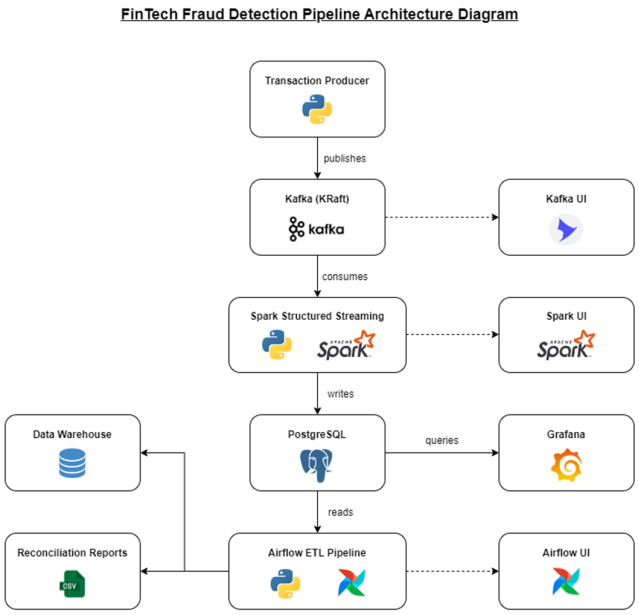

Figure 1: Pipeline Architecture Diagram

3. Tech Stack and Tool Selection

The technology stack was selected to satisfy the specified requirements which are real-time fraud detection and periodic batch reconciliation. Additionally, Docker containerization was selected in order to make the system deployable in a containerised local environment which is suitable for development environments.

a. Apache Kafka

Kafka was chosen as the message broker because it provides a durable, ordered and replayable log of transaction events. In a fraud detection context, message durability is critical because if the downstream consumer fails Kafka retains all messages and allows the consumer to resume from its last committed offset without suffering from a data loss. Kafka's KRaft mode is used for the project which eliminates the ZooKeeper dependency. It simplifies the deployment to a single container while retaining the same functionality. The “transactions” topic acts as the central ingress point for all raw financial events entering the pipeline.

b. Apache Spark

Apache Spark was selected for processing and filtering the transactions. Spark structured streaming acts as the speed layer which allows real-time processing and continuous ingestion. Additionally, stateful processing is utilized in implementing the fraud logic which allows stateful fraud detection through its applyInPandasWithState API. Stateful processing is essential for fraud detection where implementing the impossible location type fraud transaction detection rule requires knowledge of a user's previous transaction location and timestamp. In this approach which uses applyInPandasWithState API, the state store is managed by Spark. This approach provides a fault-tolerant and efficient state management without requiring an external state management system compatible with PySpark.

c. Apache Airflow

Airflow was selected as the platform to orchestrate the batch ETL layer which provides a declarative DAG-based approach to scheduling. The ETL pipeline runs every six hours as specified in the requirement which consist of tasks for reading validated non-fraud transactions from PostgreSQL for the duration of specified time window, writing them to Parquet files in order to store validated data in a data warehouse and generating reconciliation and analysis reports in CSV format. Airflow's XCom mechanism is used to pass computed statistics between ETL tasks where it facilitates using previously computed statistics when running the task related to generating the reconciliation report without re-querying the database.

d. PostgreSQL

PostgreSQL serves as the serving layer that receives transactions flagged as either "NORMAL" or "FRAUD" from the Spark streaming job via JDBC. The choice of using a relational database like PostgreSQL over a NoSQL database was taken because the structured schema of transaction records maps naturally to a relational table and SQL queries are well-suited for carrying out operations related to the domain where the data is more structured. Additionally, performing operations such as aggregation and filtering which are required by the Airflow ETL process is easier when using SQL queries.

e. Grafana

Grafana was used for monitoring and visualisation in order to facilitate a platform to observe the functionality of the fraud detection pipeline. Grafana provides tools to create live dashboards where we utilized it to create a dashboard that displays transaction throughput over time, fraud attempt counts by merchant category and the total ingress versus validated transaction amounts within a selectable time window. This allows users to directly observe the analytical results generated by the pipeline while ensuring the proper functionality of the pipeline.

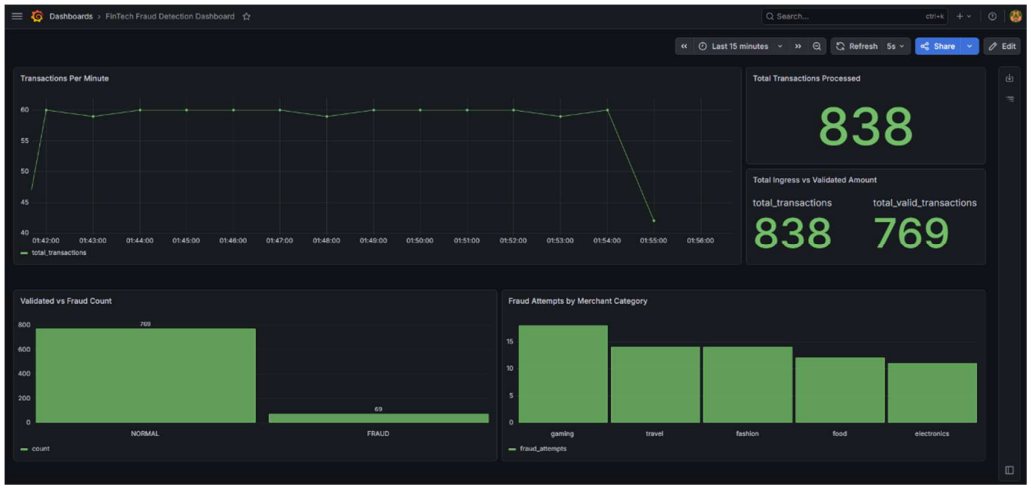

Figure 2: Grafana Dashboard

4. Event Time vs Processing Time

In stream processing, event time refers to the timestamp embedded in the event here in this case is the timestamp of the transaction. This represents the time when the transaction actually occurred. Processing time refers to the time at which the event arrives at and starts being processed by the streaming system. These two times can differ significantly due to factors such as network latency, producer delays and system backpressure.

In this pipeline, the fraud detection logic operates on event time. The timestamp field in each Kafka message represents the time at which the financial transaction occurred. This field is used directly in the impossible location type fraud transaction detection rule to compute the elapsed time between consecutive transactions for the same user. This approach for fraud detection facilitates the feature where a transaction arriving late at the streaming system will still be evaluated against the user's previous transaction time, not against the time that the transaction arrives for the processing.

The stateful processor retains the timestamp of each user's last performed transaction which is the event time in Spark's managed state store. When a new transaction arrives for a user, the time difference is computed as the number of minutes elapsed between the incoming event's timestamp and the stored previous event timestamp. This calculation is entirely independent of processing time which ensures that the fraud rule produces consistent results regardless of time when the event is consumed from Kafka. This is the best approach especially for a real time fraud detection system.

Processing time only plays a role in Spark's micro-batch trigger interval. This controls how frequently Spark polls Kafka for new messages and processes accumulated events. This doesn't influence the fraud detection logic implemented in the pipeline. The choice of processing time for the trigger is considered appropriate during the designing of the pipeline as the pipeline doesn't use watermarks or time-windowed aggregations approaches that would require a strict event time ordering method. Here in our case such a method is not required due to the above explained design choices used when implementing the fraud detection logic.

5. Ethics and Privacy Implications

Financial transaction data is considered as one of the most sensitive categories of personal information. Each record in this pipeline contains a user identifier, transaction amount, merchant category, timestamp and geographic location. In combination these fields can reveal detailed patterns about an individual's behaviour, lifestyle and financial conditions. The continuous, real-time nature of the pipeline raises concerns regarding this sensitivity where such a streaming system can process and store long records over a time period of every transaction as it occurs which creates a comprehensive surveillance profile over time for a particular user.

Several data governance principles should be applied to a production deployment of this system in order to ensure that the system adheres to the existing data governance laws and guidelines. Data minimisation requires that only the fields strictly necessary for fraud detection are ingested and stored where the fields that are not required by any fraud detection rule should be

dropped at the Kafka consumer stage before reaching PostgreSQL database. Purpose limitation requires that the transaction data collected for fraud detection should not be reused for unrelated purposes such as marketing, credit scoring without the explicit consent of the users.

Access control should be implemented to safeguard confidentiality and integrity by ensuring that the PostgreSQL serving layer, Grafana dashboard and Parquet data warehouse are accessible only to authorised users. Additionally, in adhering and aligning the system to the General Data Protection Regulation (GDPR), the organisations intend to use this kind of fraud detection pipeline must implement a data retention policy that deletes transaction records after the period required for fraud investigation purposes, a breach notification functionality and must provide facilities that fulfil user requests related to data subject rights such as right to access and right erasure.

A real time fraud detection system might carry an ethical risk of false positives. A transaction flagged as a fraud transaction may be entirely legitimate and an automated system that acts on these flags without human review could result in unjustified account suspension or payment blocking. In a production system, fraud flags should be treated as signals for human review rather than automatically enforce actions.

6. Conclusion

The FinTech Fraud Detection Pipeline demonstrates a practical application for a real-time financial big data engineering problem. Kafka provides a robust ingestion layer, Spark Structured Streaming delivers low-latency stateful fraud detection operating on event time, PostgreSQL serves as the serving layer and Airflow orchestrates periodic analysis, reconciliation and data warehousing. The technology stack was chosen by considering the specific requirements of the fraud detection scenario including the need for stateful processing and periodic reporting. The ethical considerations outlined above highlights significant privacy responsibilities related to fraud detection system that must be addressed when moving to a real-life production deployment.